API Debugging — Worked Example

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Solution

One scenario, followed across all 4 rounds: a teammate's modified copy of Doc02's `http_client.py` — the `request_with_retry()` wrapper every later document, including OpenAI calls, calls through. Work through the rounds in order. Each ends with **"What do you think is wrong?"** — stop and actually answer before reading on.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent

Round: Basic

Setup: a teammate changed the retry wrapper in their copy of `http_client.py`, making the retry/timeout settings keyword-only:

```
def request_with_retry(method, url, *, max_attempts=3, timeout=(3, 5)):
    ...
```

A caller in another file uses it like this:

```
response = request_with_retry("GET", url, 3)
```

Symptoms: the call fails immediately, before any network request is even attempted — every single time, not intermittent.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/lookup.py", line 12, in fetch
    response = request_with_retry("GET", url, 3)
TypeError: request_with_retry() takes 2 positional arguments but 3 were given
```

Expected vs. actual:

- Expected: `request_with_retry("GET", url, max_attempts=3)` runs the request, retrying as needed.
- Actual: Python refuses to even start the call.

What do you think is wrong?

Round: Intermediate

Setup: the call-signature bug is fixed. Now Doc04's OpenAI wrapper calls through `request_with_retry()` too, since it's meant to be reused everywhere. When all retries are exhausted on a bad response, `request_with_retry()` tries to build a clear error message using the failed response's body:

```
if attempt == max_attempts:
    error_body = response.json()
    raise TransientHTTPError(response.status_code, error_body)
```

Symptoms: this looks like a bug in the OpenAI response-parsing code (Doc04's layer), because that's where the traceback surfaces. It only happens when the OpenAI API is having a bad moment.

Error output:

```
Traceback (most recent call last):
  File "openai_wrapper.py", line 28, in ask
    response = request_with_retry("POST", OPENAI_URL, json=payload)
  File "http_client.py", line 51, in request_with_retry
    error_body = response.json()
  File ".../requests/models.py", line 978, in json
    raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

Expected vs. actual:

- Expected: after retries are exhausted on repeated 5xx responses, `request_with_retry()` raises a clear `TransientHTTPError` naming the status code.
- Actual: it crashes with an unrelated `JSONDecodeError` *while trying to build that very error message* — before `TransientHTTPError` is ever raised.

What do you think is wrong?

Round: Real-world

Setup: the error-message bug is fixed too. The teammate's copy wrote its own backoff line, instead of using Doc02's `compute_backoff_delay()`:

```
time.sleep(2 ** attempt)
```

Symptoms: only sometimes happens — specifically, when several callers hit a rate limit at close to the same moment, which single-caller manual testing never reproduces.

Log output:

```
[10:02:01] agent=research attempt=1 got 429, sleeping 1s
[10:02:01] agent=analysis attempt=1 got 429, sleeping 1s
[10:02:01] agent=writer   attempt=1 got 429, sleeping 1s
[10:02:02] agent=research attempt=2 got 429, sleeping 2s
[10:02:02] agent=analysis attempt=2 got 429, sleeping 2s
[10:02:02] agent=writer   attempt=2 got 429, sleeping 2s
[10:02:04] agent=writer   attempt=3 got 429, sleeping 4s
[10:02:04] agent=research attempt=3 got 429, sleeping 4s
[10:02:08] agent=writer: TransientHTTPError after 4 attempts
```

Expected vs. actual:

- Expected: callers that hit a shared rate limit at the same moment spread their retries out, so at least one of them gets through soon after the limit clears.
- Actual: every caller sleeps the exact same fixed durations (1s, 2s, 4s, 8s…), so they keep retrying in lockstep and colliding with the same limit, again and again — reproducible only when multiple callers are actually hitting the limit together, not with one caller alone.

What do you think is wrong?

Round: Multi-agent

Setup: jitter got added, and `request_with_retry()` now has a real max-attempt cap — but no cap on *total wait time* across all those attempts. In Project 4, the Research agent's search tool calls through this same `http_client.py`.

Symptoms: a unit test of `request_with_retry()` alone, using a fake transport that never actually sleeps, finishes in milliseconds and passes cleanly. Running the full 4-agent pipeline against a genuinely degraded upstream API looks completely different — the Supervisor's routing log shows a long silent gap right after handing off to Research, and it's tempting to assume the graph itself is stuck (a LangGraph-shaped symptom).

Log output:

```
[10:14:02] supervisor: routing to research_agent
[10:14:02] research_agent: calling search tool
... (no further log lines for 31 seconds) ...
[10:14:33] research_agent: search tool succeeded on final attempt
[10:14:33] supervisor: routing to analysis_agent
```

Expected vs. actual:

- Expected: if the underlying API is unrecoverable, the pipeline fails fast with a clear error, instead of the Supervisor and everyone watching the run assuming something in the graph itself is hung.
- Actual: `request_with_retry()`'s backoff runs its full course (1+2+4+8+16 = 31s) silently, inside one single tool call the Supervisor has no visibility into — a fixed retry-count cap did nothing to bound how long that felt like a hang, because it never bounded *total* time, only attempt count.

What do you think is wrong?

Full solution: Show me the solution